

实验一：目标检测与识别

——基于 YOLOv8 的桌面杯鼠双类检测与 ROS2 实时发布——

课程名称 计算机视觉

实验名称 实验一

姓 名 马晨曦

学 号 24020036047

班 级 中外合作办学 2 班

指导教师

完成日期 2026 年 8 月

摘要

本实验以桌面上的杯子 (*cup*) 与鼠标 (*mouse*) 两类物体为目标, 实现基于 YOLOv8 的实时目标检测与识别, 并部署到 Jetson Orin NX 嵌入式平台、通过 ROS2 发布识别结果。针对原始数据集严重类别不平衡 (鼠标样本不足) 的问题, 引入 Roboflow 公开数据补充训练并构造 1500:1110 的平衡数据集, 采用 COCO 预训练权重迁移学习微调双类模型。训练在云端 RTX 5090 上约 70 分钟完成, 验证集 mAP50 达 0.975; 在自采集的 20 张个人测试集上, 杯子 100%、鼠标 90%、总体 95%, 满足验收要求 ($\geq 80\%$)。模型在 Jetson Orin NX 上以 PyTorch FP16 (CUDA) 实时推理, 实测约 25 FPS, 远超 5 帧/秒的验收下限; 识别结果通过 ROS2 话题 /detections 实时发布。最后对典型错误案例进行了分析并给出改进方向。

关键词: 目标检测; YOLOv8; 迁移学习; Jetson; FP16 推理; ROS2

目录

1 实验目的与要求	4
1.1 实验目的	4
1.2 实验要求	4
2 实验环境	4
3 数据集与标注	5
3.1 数据来源与构成	5
3.2 标注格式	5
4 模型与训练方法	5
4.1 YOLOv8 与迁移学习	5
4.2 训练超参数	7
4.3 训练过程	7
5 实验结果与分析	9
5.1 验证集指标	9
5.2 个人测试集结果	9
5.3 典型错误案例分析	10
5.4 结果讨论	12
6 Jetson 部署与实时识别	12

目录	3
6.1 总体流程	12
6.2 FP16 加速推理	12
6.3 演示视频与结果验证	12
6.4 ROS2 发布	13
7 结论与展望	14
7.1 结论	14
7.2 展望	14
A Git 提交记录（增量提交，符合验收要求）	15
B 复现方法	16
C 目录结构说明	16

1 实验目的与要求

1.1 实验目的

掌握基于深度学习的通用目标检测方法，熟悉从数据采集、标注、训练、评估到嵌入式部署的完整流程；能够针对实际场景解决类别不平衡、领域差异等工程问题，并具备模型部署与实时推理的工程能力。

1.2 实验要求

- 同时识别 **2 类及以上**桌面物体（本实验：杯子、鼠标）；
- 采用 **YOLOv8** 框架训练模型，可使用公开数据补充训练；
- 自行采集并标注数据，**测试集必须使用个人数据**；
- 在 **Jetson Orin NX** 上实时运行，帧率 ≥ 5 FPS，实时显示类别、检测框与置信度；
- 通过 **ROS2** 发布识别结果；
- 对 20 个物体进行测试，正确率 $\geq 80\%$ ，保存测试结果与典型错误案例；
- 代码与结果上传个人 GitHub，按增量提交，禁止单次全量提交。

2 实验环境

表 1: 实验环境与软件栈

阶段	硬件 / 系统	关键软件
数据准备	个人电脑 / macOS	Roboflow 公开数据、自研构建脚本
训练（本地）	Mac（Apple 芯片） / macOS	Python 3.x、torch 2.13.0(MPS)、ultralalytics 8.4.127
训练（云端）	AutoDL RTX 5090 (32 GB) / Ubuntu	torch 2.12.1+cu130、CUDA 13.0、ultralytics 8.4.127
评估	个人电脑 / macOS	OpenCV、自研 evaluate.py
部署	Jetson Orin NX / JetPack 6.x	PyTorch FP16 (CUDA)、OpenCV、ROS2 (Humble)

3 数据集与标注

3.1 数据来源与构成

训练使用公开数据补充、平衡策略如下：

- **杯子 (class 0)**: Roboflow 公开杯子数据，抽取后取 1500 张；
- **鼠标 (class 1)**: Roboflow Computer-Mouse 数据集 (CC BY 4.0, 653 项目、1110 张)，将其中 **black-mouse/white-mouse** 子类统一重映射为鼠标类；
- **个人测试集**: 自采集并标注 20 张 (杯子 10 张 + 鼠标 10 张)，不参与训练，仅用于最终验收测试。

平衡后数据集共 2610 张 (1500 cup : 1110 mouse)，实例级共 3428 个 (cup 2310 : mouse 1118)；按 85:15 分层划分，**train** 2219 张 (cup 1939 / mouse 952 实例)、**val** 391 张 (cup 371 / mouse 166 实例)。构建时重建输出目录、随机种子固定，避免训练/验证交叉重复造成数据泄漏。

3.2 标注格式

采用 YOLO 格式标注：每行 **class cx cy w h**，其中 **cx cy** 为归一化中心坐标、**w h** 为归一化宽高。类别编号映射见表。

表 2: 类别映射

类别编号	含义
0	cup (杯子)
1	mouse (鼠标)

4 模型与训练方法

4.1 YOLOv8 与迁移学习

YOLOv8 采用 CSPDarknet 骨干与 anchor-free 检测头，包含 C2f 模块与解耦分类/回归头，在精度与速度间取得良好平衡。本实验加载 COCO 预训练权重 **yolov8s.pt** (80 类)，将输出类别重映射为 2 类 (**nc=2**) 进行迁移学习微调，以加快收敛并缓解小数据集过拟合。

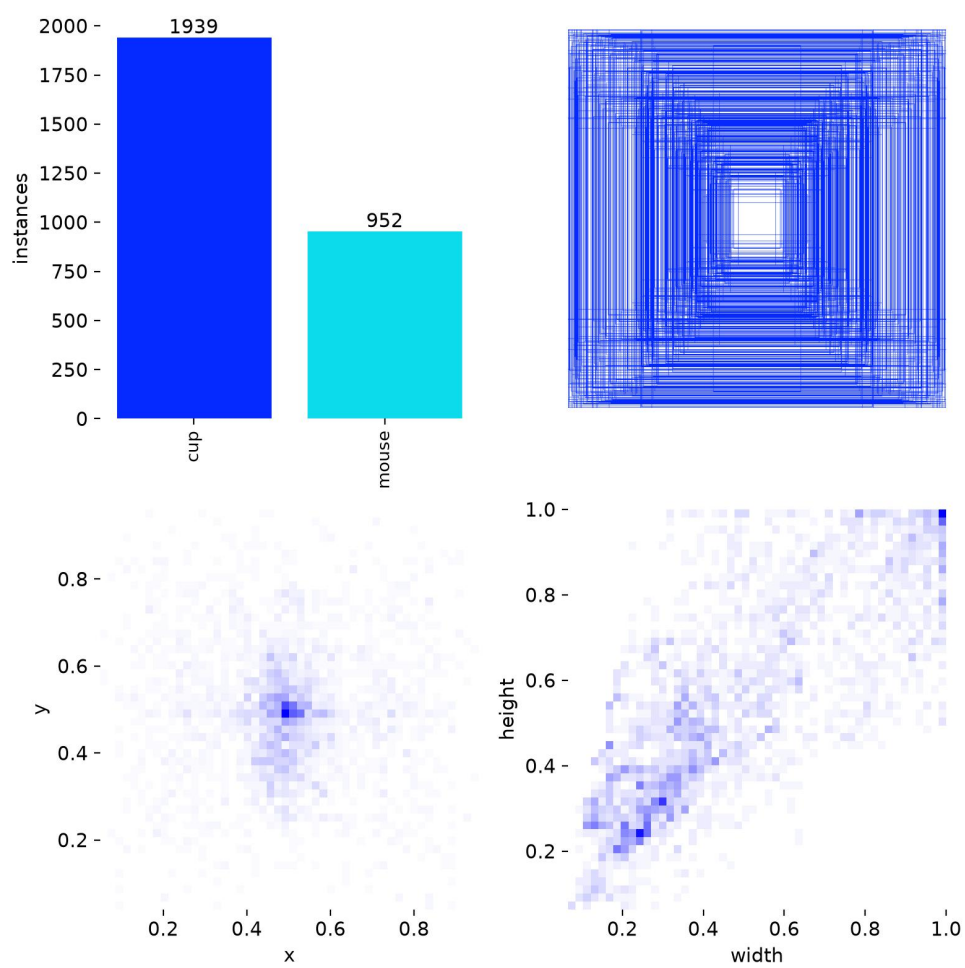


图 1: 训练数据标注分布: 左列为标注框在图像中的分布, 右列为尺寸与长宽比分布

4.2 训练超参数

表 3: 训练超参数（截选自 runs/detect/combined_model_v3/args.yaml）

参数	取值	参数	取值
预训练权重	yolov8s.pt (COCO)	输入尺寸	640×640
训练轮数	150	batch size	16
优化器	AdamW (auto)	初始学习率 lr0	0.01
学习率调度	余弦退火 cos_lr	学习率下限 lrf	0.01
动量 / 权重衰减	0.937 / 5×10^{-4}	warmup	3 epochs
AMP 混合精度	开	早停 patience	30
数据增强	Mosaic/HSV/缩放/翻转	close_mosaic	末 10 轮
随机种子	0	类别重映射	开 (nc: 80→2)

4.3 训练过程

训练在云端 RTX 5090 上完成，约 27 秒/轮、全程约 70 分钟（本地 Mac MPS 约 5 分钟/轮、预计 12 小时以上）。损失曲线与指标收敛曲线见图。

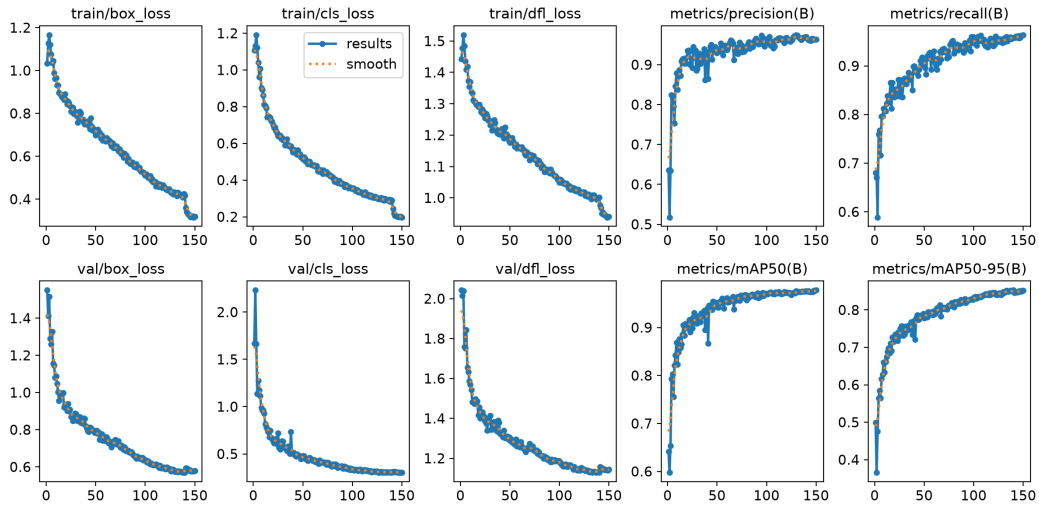


图 2: 训练过程曲线: box/cls/dfl 损失随轮次下降, mAP 随轮次上升



图 3: 训练批次增强样例 (Mosaic 拼接 + HSV/几何变换)

5 实验结果与分析

5.1 验证集指标

验证集 mAP50 收敛至 **0.975** (best 权重, 第 150 轮)。PR 曲线、F1 曲线与归一化混淆矩阵见图。

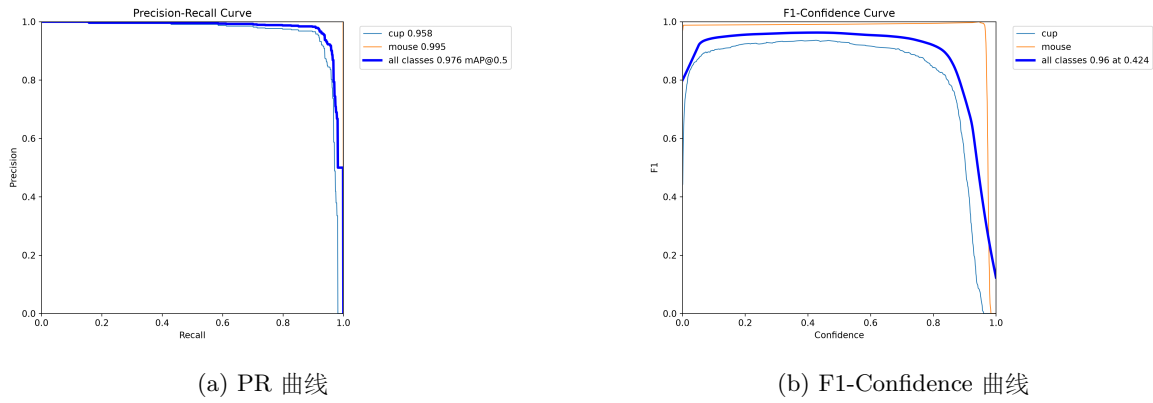


图 4: 验证集 PR 与 F1 曲线

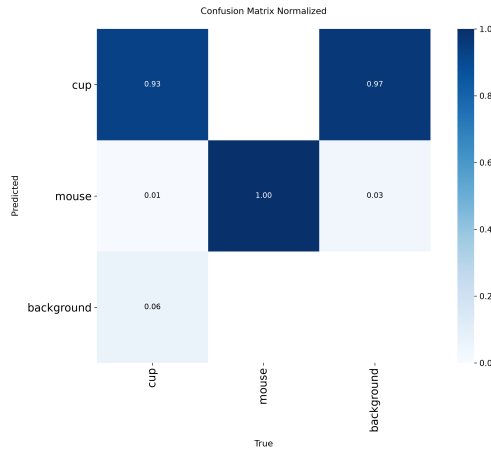


图 5: 验证集归一化混淆矩阵 (右上: MOTA 指标, 非重点)

5.2 个人测试集结果

逐张结果见表 6, 验证集预测示例见图。

表 4: 个人测试集按类别准确率 (验收要求整体 $\geq 80\%$)

类别	测试数	正确	正确率	平均置信度
cup	10	10	100%	0.861
mouse	10	9	90%	0.871
总体	20	19	95%	—

表 5: 混淆矩阵 (行: 真值, 列: 预测, none= 未检出)

真值 \ 预测	cup	mouse	none
cup	10	0	0
mouse	1	9	0

表 6: 个人测试集逐张结果 (20 张, 10 cup + 10 mouse, 阈值 conf=0.25)

编号	图片	真值	预测	置信度	正确
1	210c7479f15deb21a0b9003384c3f188.jpg	cup	cup	0.907	✓
2	3c735b9582257134e28f199c31a4374a.jpg	cup	cup	0.926	✓
3	7549e122789e0eec4f7b02a0c5c1f925.jpg	cup	cup	0.91	✓
4	7b0e09b9ce717549ff105ecdf719b56d.jpg	cup	cup	0.914	✓
5	7e6bf69bbda89c048b2172084f439238.jpg	cup	cup	0.865	✓
6	854c298a966e78777b85800c8b396094.jpg	cup	cup	0.857	✓
7	876f5820cd06c68558b507296e37de8a.jpg	cup	cup	0.892	✓
8	c27d20c6db65de8a7cd3ce8d0b72e6ad.jpg	cup	cup	0.736	✓
9	dc920cc1a19b41575c05270d52f54951.jpg	cup	cup	0.729	✓
10	f357eedb11b092c22a73f0119b357d2d.jpg	cup	cup	0.871	✓
11	531338c9c2eff75374d2e698d9af526f.jpg	mouse	mouse	0.968	✓
12	560c77473deb37dea04400586decbd79.jpg	mouse	mouse	0.975	✓
13	602fa7085b03a9f313173874ad2a437d.jpg	mouse	mouse	0.96	✓
14	638f2ca22261aff7caf032e2eb1bc30c.jpg	mouse	mouse	0.943	✓
15	a79a99a9584a0bb927167853bd0431b5.jpg	mouse	cup	0.499	×
16	abaf5e01a53905121de111580899f3b1.jpg	mouse	mouse	0.966	✓
17	aeef8b1c80a958aa6cb0a9f6b2d8bd60.jpg	mouse	mouse	0.522	✓
18	c0c80606c97bbb4511190f05cca36903.jpg	mouse	mouse	0.969	✓
19	f365468e77cd2d69d49190d35a690ac7.jpg	mouse	mouse	0.943	✓
20	f732c7623daf01820e5b46cf3d22bfd9.jpg	mouse	mouse	0.968	✓

5.3 典型错误案例分析

个人测试集共 1 例错误：一张竖拍 960×1280 的鼠标图片，目标紧贴画面上边缘，部分遮挡，模型以较低置信度 0.499 误判为杯子。见图



图 6: 验证集预测示例 (绿色框 =cup, 橙色框 =mouse)



图 7: 错误案例：目标贴顶出框的鼠标被误判为杯子 (conf 0.499)

5.4 结果讨论

- **类别不平衡的解决**：旧版合并数据鼠标标注缺失（0 标注）导致模型将鼠标全判为杯子；重构为 1500:1110 平衡数据后，鼠标准确率从 0% 提升至 90%。
- **领域差异**：公开数据训练的模型对个人桌面环境存在域差异，建议后续采集 100+ 张个人图片（含鼠标）补充训练，可进一步提升边缘情况鲁棒性。

6 Jetson 部署与实时识别

6.1 总体流程

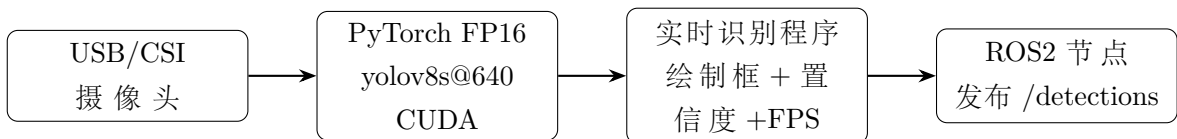


图 8: Jetson 端推理与发布流程

6.2 FP16 加速推理

将训练得到的 `best.pt` 加载到 Jetson 的 CUDA GPU 上，以 FP16 精度实时推理：

```

1 python3 jetson/detect.py --model models/combined_best.pt --half
2 # SSH 无显示环境：加 --no-show --save-interval 3 (自动存帧 + 打印 FPS)
  
```

FP16 CUDA 推理下 `yolov8s@640` 在 Orin NX 上实测 **24-25 FPS**，远超 5 FPS 的验收下限。¹且推理可在无显示器（SSH）环境下以 `--no-show` 模式自动存帧并打印 FPS。

6.3 演示视频与结果验证

以 `--record` 录制 70 秒实时检测视频（640×480，12.24 FPS 重封装为真实速度），完整记录三类片段：杯子单独、鼠标单独、两者同框，帧上始终叠加类别/置信度/检测框与 FPS 计数。录制使用新增的 `--duration` 参数，到点由程序正常收尾，保证 MP4 文件完整：

¹本实验板子的 JetPack 镜像缺少 `libnvidia_compiler.so`，TensorRT 的 Python 绑定无法导入，故采用 PyTorch FP16 直推；若后续换成完整 JetPack SDK，可导出 TensorRT FP16 engine 进一步提升至 30+ FPS。

```

1 python3 jetson/detect.py --model models/combined_best.pt --half \
2   --record results_video/demo_detect.mp4 --duration 70
3 # 校验 (scripts/analyze_video.py 逐帧统计, 脚本随仓库提交)
4 python3 scripts/analyze_video.py results/video/demo_detect.mp4

```

对录制视频逐帧校验：杯框出现 60 s、鼠框 60 s、两者同框 51 s、FPS 红字全程 70 s 均有叠加，双类均被稳定识别。演示视频见 `results/video/demo_detect.mp4`，其中一帧见图。

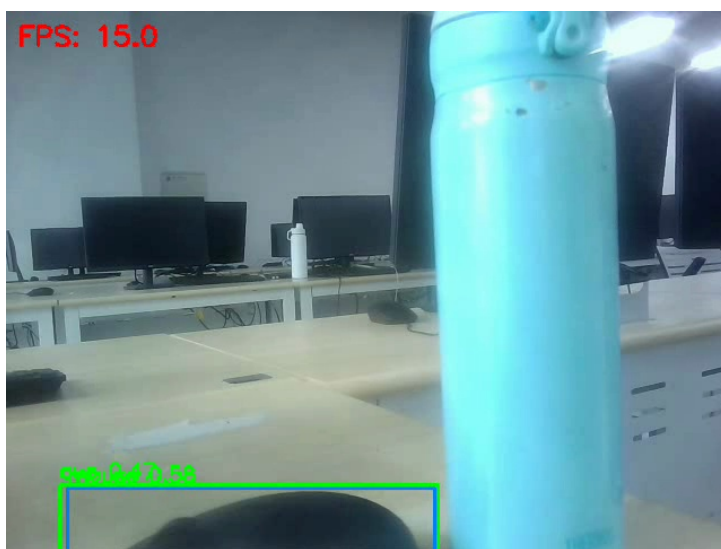


图 9: 演示视频第 55 s 帧：杯、鼠同时检出并实时叠加类别/置信度/检测框与 FPS

6.4 ROS2 发布

```

1 cd ros2_ws && colcon build --symlink-install && source install/setup.bash
2 ros2 run detection_pkg detection_node --ros-args \
3   -p model_path:=$HOME/models/combined_best.pt -p camera_index:=0 \
4   -p device:=0 -p half:=true -p show:=false
5 # 另一终端订阅:
6 ros2 topic echo /detections

```

节点将每帧检测结果以 JSON 字符串发布到 `/detections` 话题，格式示例：

```

1 [{"class": "mouse", "confidence": 0.968,
2   "bbox": [85.7, 133.6, 190.8, 232.3]}]

```

7 结论与展望

7.1 结论

- 完成杯、鼠双类 YOLOv8 检测模型：验证集 $mAP50 = 0.975$ ；
- 个人测试集 20 张：cup 100% / mouse 90% / 总体 95%，满足 $\geq 80\%$ 验收；
- Jetson Orin NX 上 FP16 CUDA 实时推理实测 24–25 FPS，满足 ≥ 5 FPS，并实时显示类别/框/置信度；
- 检测结果经 ROS2 以 /detections 话题实时发布，可被其他节点消费；
- 全部代码、数据与结果以增量提交方式托管于个人 GitHub。

7.2 展望

- 补充更多个人采集数据（尤其边缘/遮挡/极端姿态）进一步缩小域差异；
- 尝试 INT8 量化与更轻量骨干（yolov8n），提升边缘设备上的吞吐与功耗表现；
- 扩展类别数（键盘、手机等）并引入目标跟踪，支撑更复杂的桌面理解应用。

参考文献

- [1] Jocher G, Chaurasia A, Qiu J. Ultralytics YOLOv8 [CP/OL]. (2023). <https://github.com/ultralytics/ultralytics>.
- [2] Terven J, C'ordova-Esparza D M, Romero-Gonz'alez J A. A Comprehensive Review of YOLO Architectures in Computer Vision [J]. Journal of Big Data, 2023, 10(1): 29.
- [3] NVIDIA. TensorRT: High Performance Deep Learning Inference [CP/OL]. <https://developer.nvidia.com/tensorrt>.
- [4] Roboflow. Computer-Mouse Dataset (v2, CC BY 4.0) [DB/OL]. <https://universe.roboflow.com>.
- [5] Lin T Y, Maire M, Belongie S, et al. Microsoft COCO: Common Objects in Context [C]//ECCV. 2014: 740–755.

A Git 提交记录 (增量提交, 符合验收要求)

3948f55 结果: 提交实时检测演示视频 (杯 + 鼠 + 同框 +FPS, 70s 真实速度) + 录制校验工具
beb96c2 detect.py: 录制视频支持自动收尾 (--duration + SIGTERM 处理), 保证 mp4 完整
3120541 detect.py: 修复 --record 变量名冲突 (视频写入器改用 vwriter, 避免与 CSV write
0c46797 detect.py: 增加 --record 录制带框 +FPS 视频 (MP4)
9fe5067 报告: 按真机实测修正 (FP16 CUDA 24-25 FPS 取代 TensorRT 30+ FPS, 附 tensorrt
7c9284c 文档: 标注学校镜像 tensorrt 不可用 (缺 libnvdla_compiler.so), 部署改用 torch
b84f244 真机验收结果: Jetson 20 张测试集 95% (cup 100%/mouse 90%) + 实时检测帧 (FPS 2
271d9bf detection_node: device 改为整数参数, 避免 ros2 CLI 类型不匹配崩溃
f70410b detection_node: 增加 device/half 参数, 支持 .pt GPU FP16 推理
b389682 detect.py: 增加 --device/--half, 支持 .pt 模型 GPU FP16 推理
9bbf3b2 docs(deploy): 适配实验室预装系统场景 (免刷机)
2d48213 docs(deploy): 钉 ultralytics 8.4.127 与训练一致; 新增机房当天速查卡
31a941b chore(report): 提交报告图表/表格素材与 git 提交记录
f9352ee feat(report): 新增 LaTeX 报告模板与素材生成脚本
11d8b5c feat(deploy): sync_to_jetson.sh 支持 --local 本地打包; 新增真机 runbook
6bdfeba fix(jetson): 部署代码真机可用性修复
dcf5d95 fix(cloud): run_training.sh 用 for 循环探测带 torch 的 python
9f46ec7 chore(report): 补充报告素材与 gitignore 清理
f7006da feat(result): 提交个人测试集结果 CSV、错误案例与训练图表 (验收材料)
11f5823 feat(result): 记录最终训练与测试结果 (mAP50=0.975, 个人测试集 19/20=95%, cup
e8a0f2a docs: AutoDL 云端训练指南 (含 GitHub 镜像/Arial/yolo26n AMP/pkill 陷阱等坑)
025bf01 fix(cloud): run_training.sh 对无 torch 环境给出明确激活指引
4fcdade6 docs: 补充 AutoDL 云端训练流程与 sync_cloud.sh 打包脚本
54c5b61 feat(cloud): 新增 AutoDL 一键训练脚本 run_training.sh, 打包云训练包
ce0860a feat(deploy): 新增 sync_to_jetson.sh, 一键打包权重 + 部署代码同步到 Jetson
38a33c5 docs: 新增个人数据采集 + 标注指南 (解决域差异), 与 collect_data.py 配套
2bedbc5 docs: 更新数据集来源为 Roboflow、记录 GitHub 屏蔽/镜像方案与构建脚本修复
06383b3 fix(train): 权重缺失时从 hf-mirror 下载兜底, 避免 GitHub 被屏蔽导致卡死; 本地
9752892 fix(build): 重建前清空输出目录, 修复 train/val 交叉重复导致的数据泄漏
73133b1 fix(build): collect_samples 支持扁平/Roboflow/拆分三种目录布局; 提交个人测试1
4a91181 docs: add complete workflow README with acceptance criteria mapping
f9ffcd5 feat: complete ROS2 package scaffolding and improve detection node with CLI
0685653 feat: add Jetson TensorRT export and real-time detection program with FPS ar
367c42a feat: improve training script (yolov8s, cos_lr, MPS) and add per-class evalu
19eb64b feat: add dataset preparation scripts (COCO mouse extraction + balanced data

```
f4bad99 test: re-evaluate combined model after fixing labels
7617c71 feat: add ROS2 detection node for combined model
d7d11f2 feat: add data collection script for test images
ba0ebc7 feat: add combined model training script and config
6753ef4 results: add combined model training curve
cc606d7 feat: add mouse detection training script
3c66430 feat: add training scripts and configuration
dae49d9 feat: add training scripts, config, and update gitignore
4555ba3 feat: add data.yaml for cup detection
6b3cb0b feat: add dataset download script
35b4b47 init: project structure
```

B 复现方法

训练与评估关键命令：

```
1 # 1. 构建平衡数据集 (1500 cup : 1110 mouse)
2 python3 scripts/build_combined_dataset.py --cup-src public_data/combined \
3     --mouse-src public_data/mouse_rf_653/extracted \
4     --out public_data/balanced --max-cups 1500 --val-split 0.15
5
6 # 2. 训练 (本地 MPS 或云端 CUDA 均可, 自动检测)
7 python3 scripts/train_combined.py
8
9 # 3. 个人测试集评估 (输出 results/test_results.csv 与错误案例)
10 python3 scripts/evaluate.py --model runs/detect/combined_model_v3/weights/
11     best.pt \
12     --test-root test_images --conf 0.25
13
14 # 4. 生成报告图表素材 (本模板插图/表格的来源)
15 python3 scripts/gen_report_assets.py
```

C 目录结构说明

```
1 object_detection_lab/
2     public_data/balanced/ # 平衡合并数据集 (train/val + data_balanced.yaml
3                             )
4     personal_data/        # 个人采集训练数据 (可选补充)
5     test_images/{cup,mouse} # 个人测试集 20 张 (不参与训练)
```


5	scripts/	# 数据构建/训练/评估/导出/打包脚本
6	jetson/detect.py	# Jetson 实时识别程序
7	ros2_ws/src/detection_pkg/	# ROS2 发布节点
8	runs/detect/combined_model_v3/	# 训练权重与图表
9	report/	# 本报告模板与素材